
REST APIs

01

Recap



02

REST w/ Django (continued)



03

REST w/ Spring boot



04

GraphQL



Agenda

Application Architecture

Patterns: MV*

Common Terms

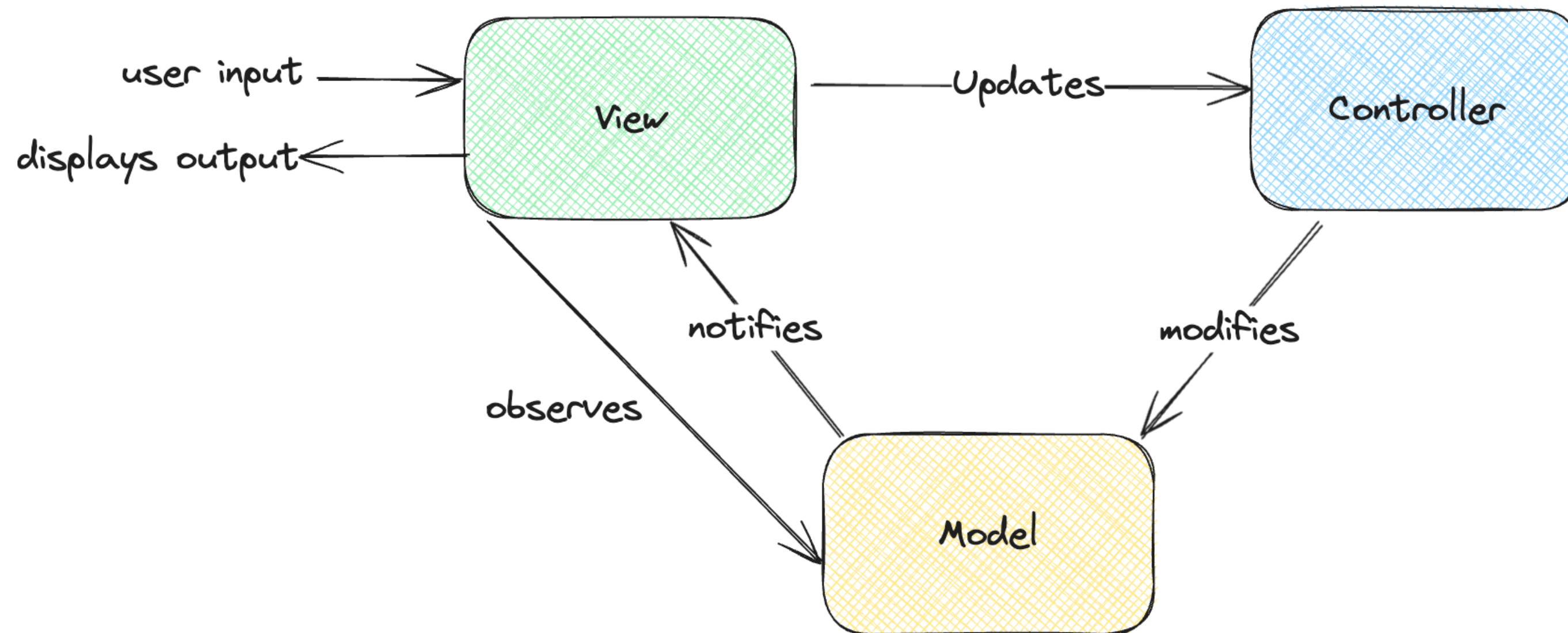
Model - manages data for an app

View - visual representation of the model

Application Architecture Patterns: MV*

Model View Controller (MVC)

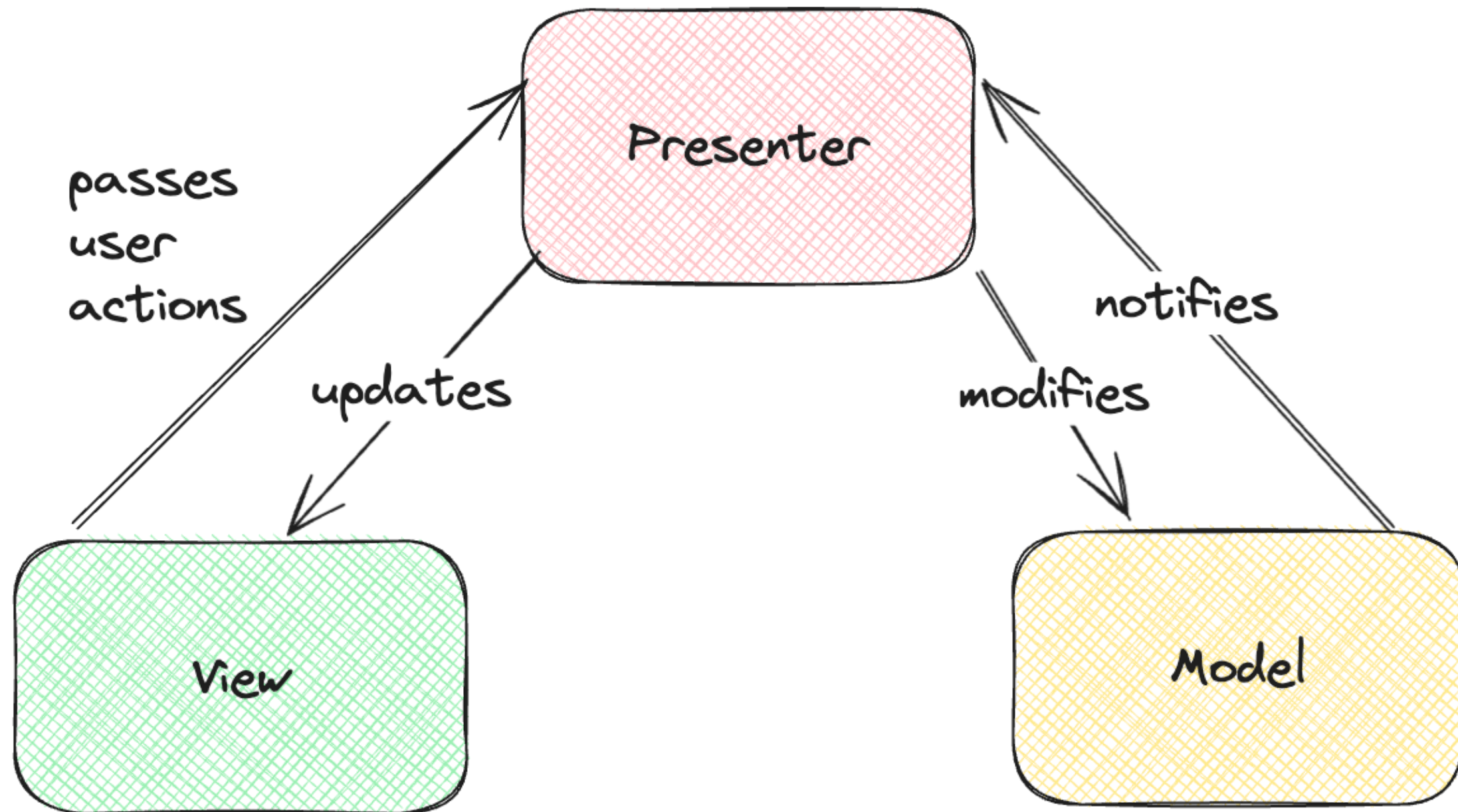
Controllers - in charge of updating the model



Application Architecture Patterns: MV*

Model View Presenter

Presenter - handles modifying the model and the view

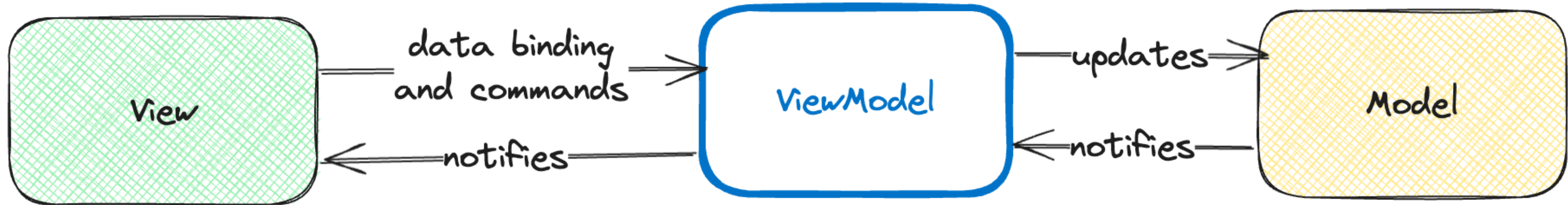


Application Architecture Patterns: MV*

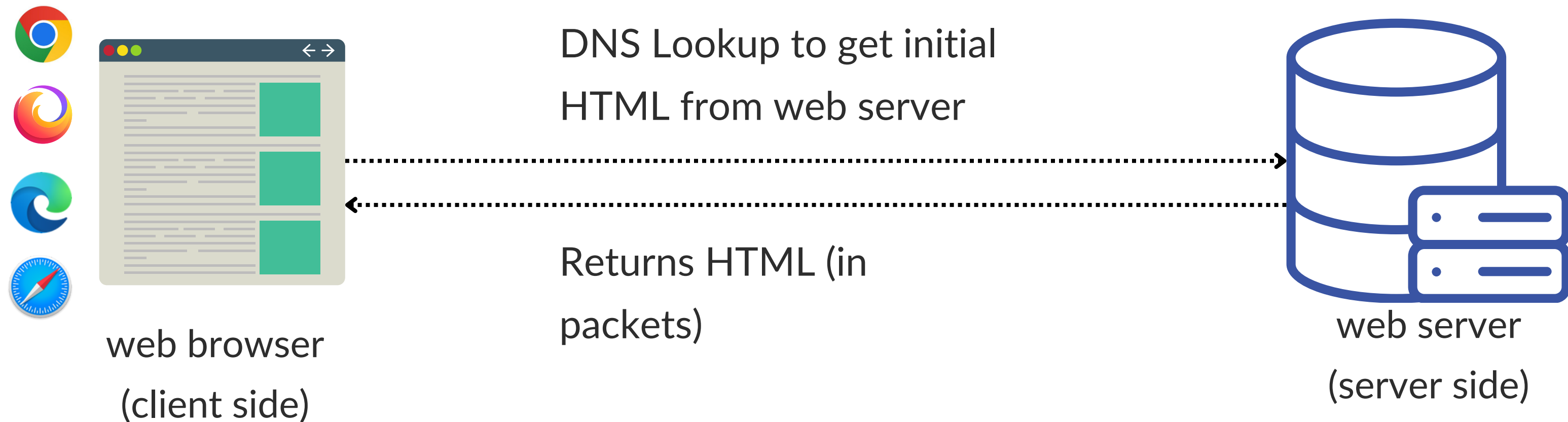
Model View ViewModel

ViewModel data converter with methods exposed to view

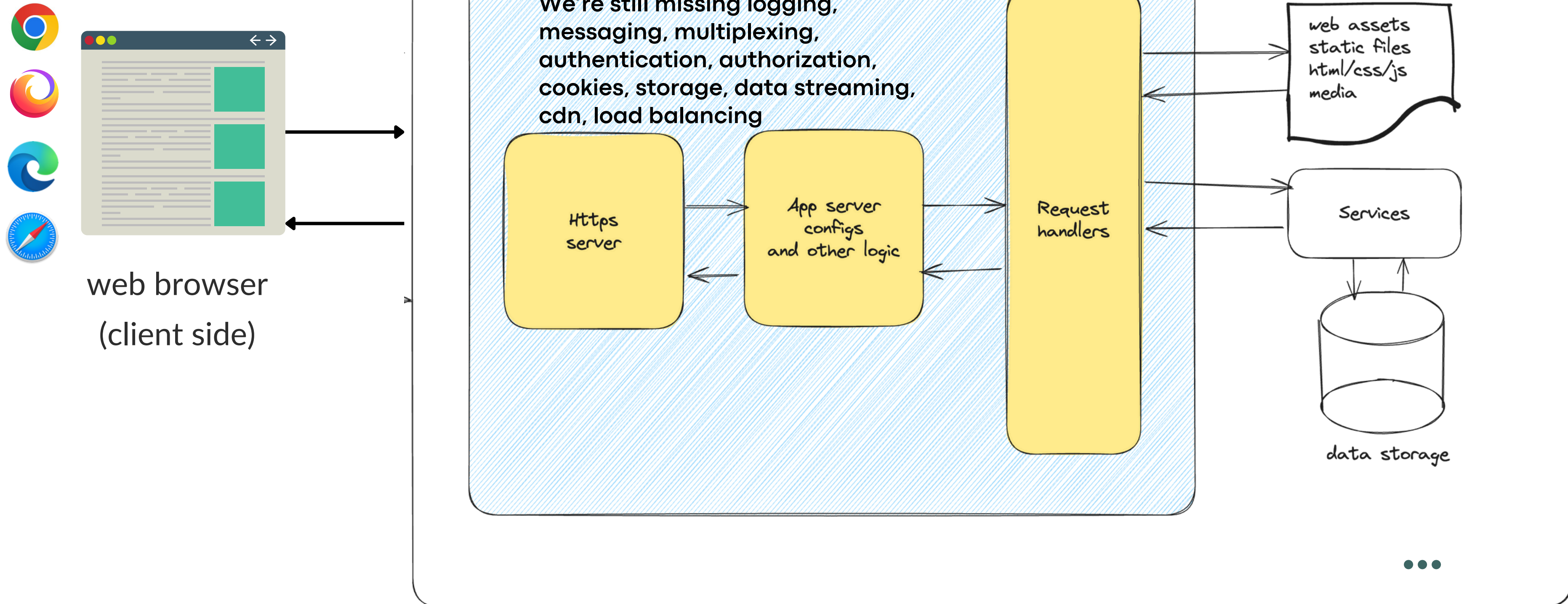
View is active



The web app that we all know...



The web app that we all know...



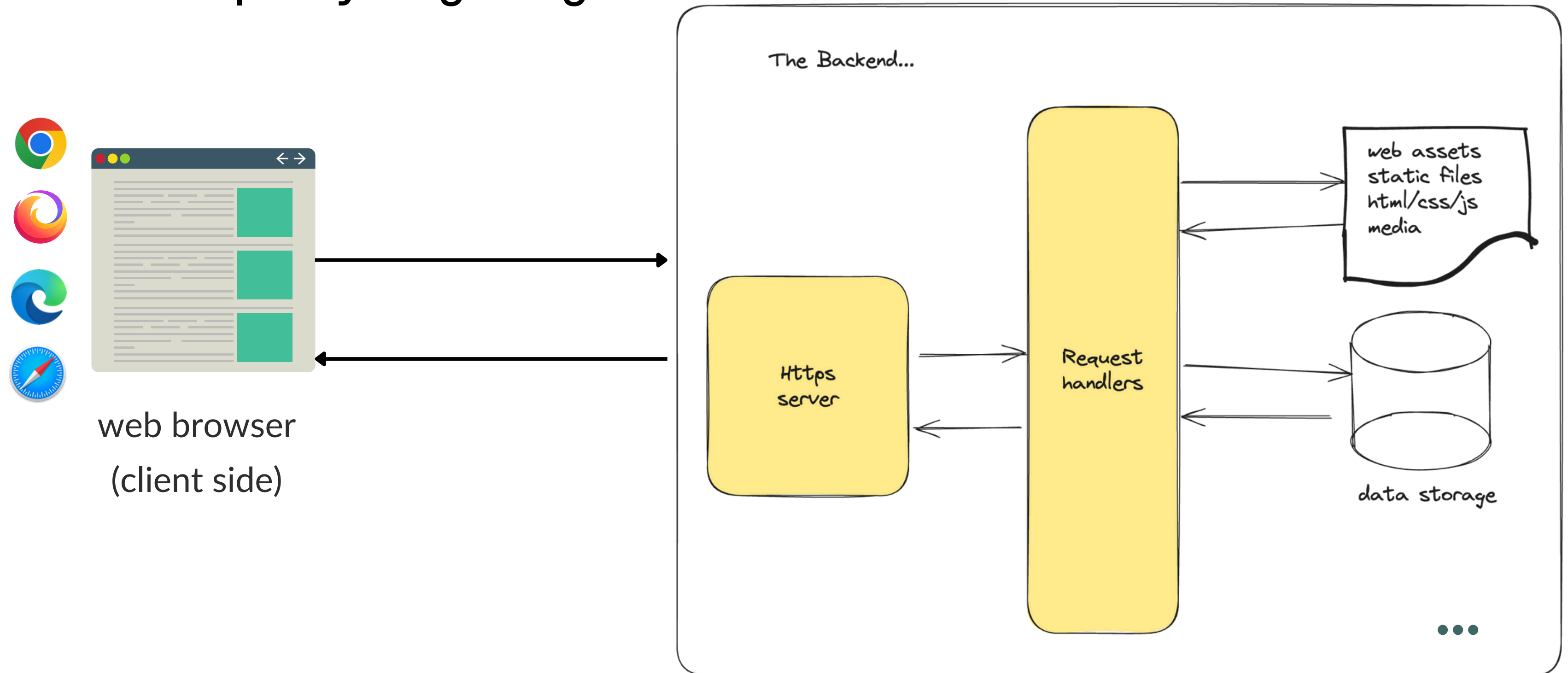
Architecture Patterns for the BE

- SPA (Thin server architecture)
- Microservices
- Serverless
- Layered
- Event based/driven* (we'll discuss next week with sockets)

Patterns for the BE

SPA (Thin server architecture)

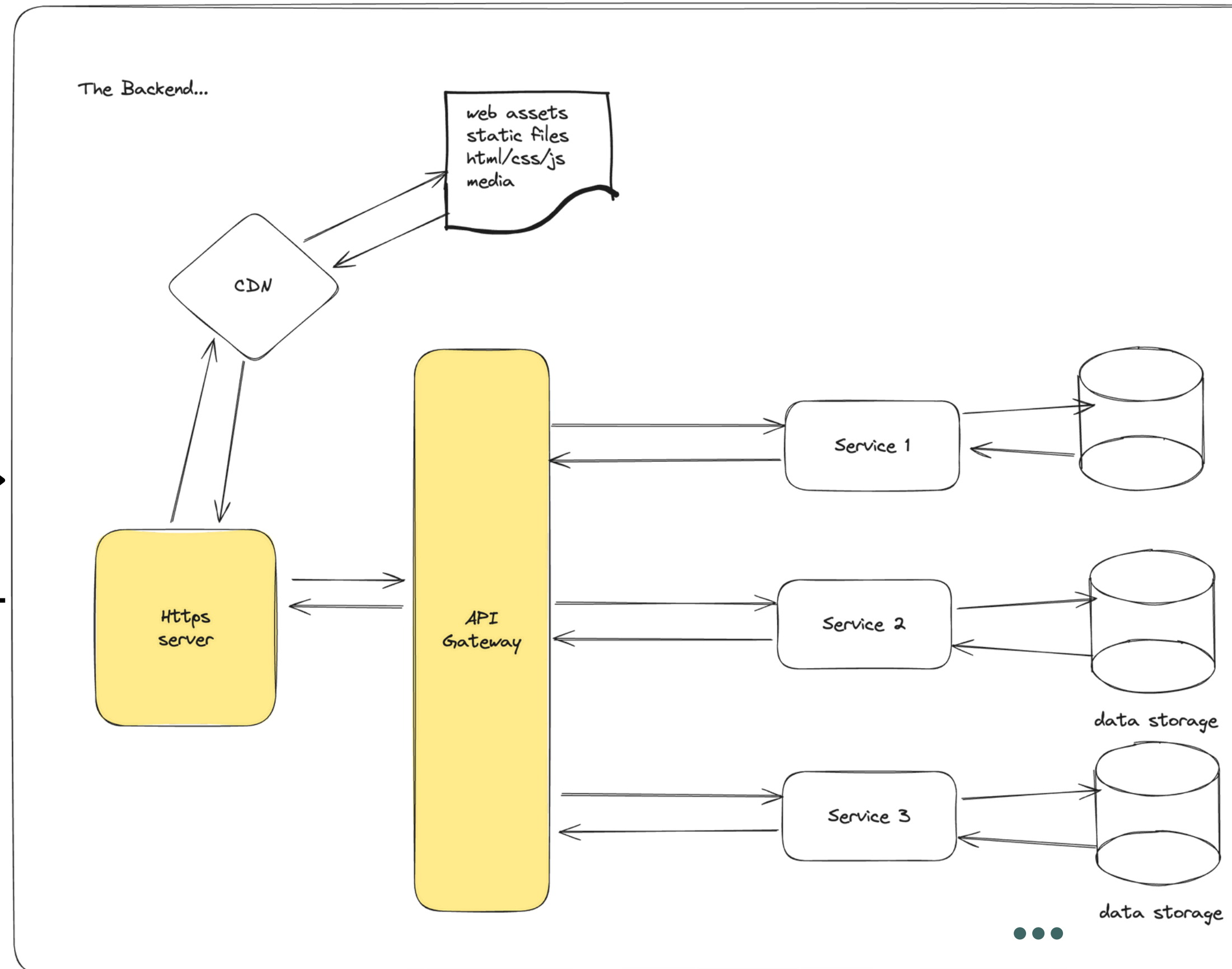
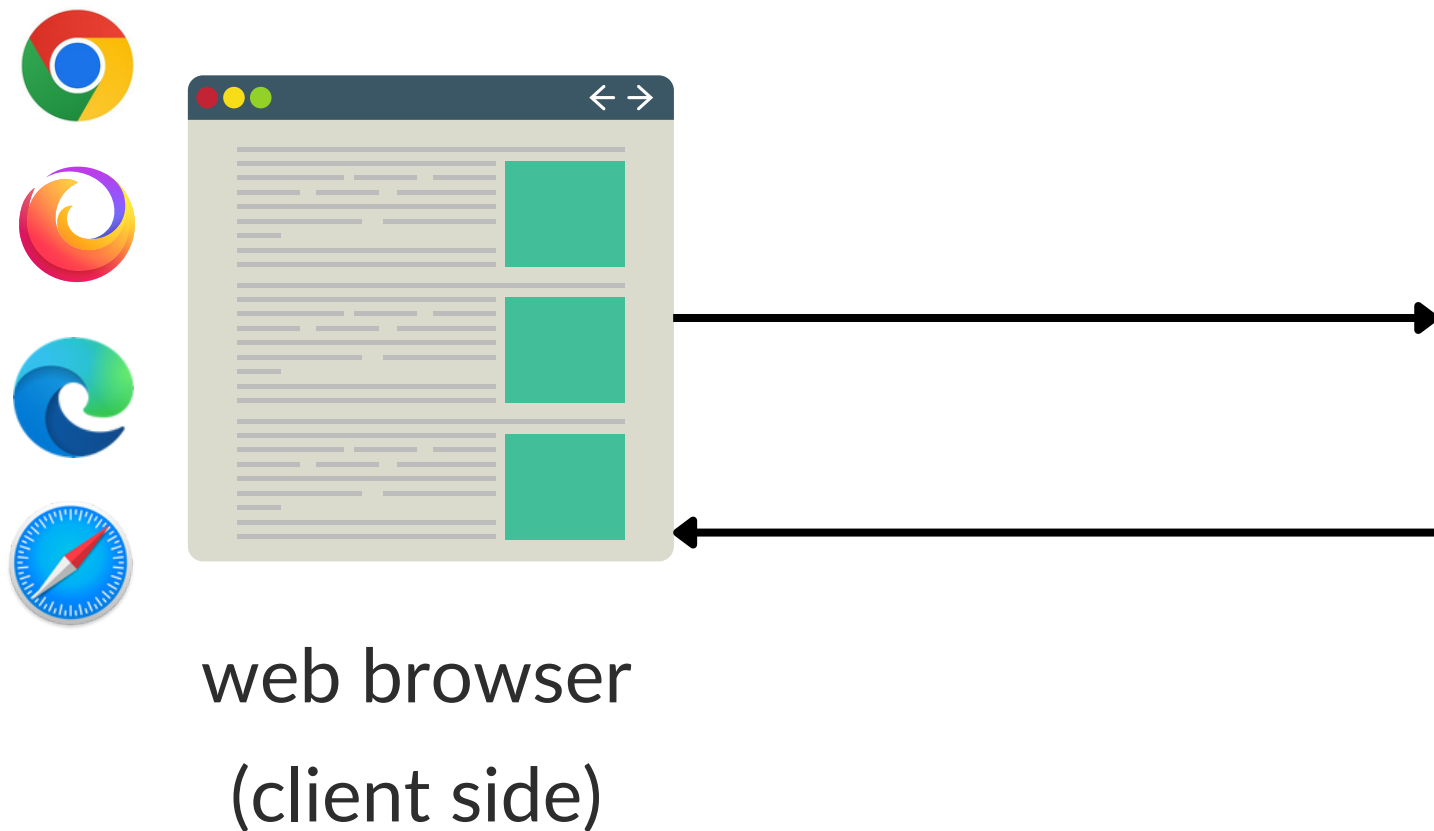
- client performs a lot of the business logic
- server is purely for getting data



Patterns for the BE

Microservices

- When you break up a large app into smaller independent pieces
- Each service has one purpose



Frameworks for the BE

- Django (Python)
- Express (JS)
- Spring boot (Java)
- Ruby on Rails (Ruby)

Key Components of a BE Framework

- Routing
- Authentication/Authorization
- Template engine (though this can usually be handled by FE framework)
- Db integration

API FORMATS

- SOAP
- gRPC
- GraphQL
- REST(ful)
- WebSocket
- Webhooks

What is REST?

Representational State Transfer - architectural style for APIs (not a protocol)

Representational - resource is a representation of what the server side has

State - client requests move server through different states

Transfer - result of state transition gets returned to the client

REST(ful)

- Representational State Transfer

Constraints:

1. Uniform Interface
2. Stateless Operations
3. Layered System
4. Client-server based
5. RESTful resource caching
6. Code on demand

RESTful APIs adhere to all REST constraints



Anatomy of a Request

Includes:

Request method

Doc path

Http version

Additional info for what type of data is being requested (ex: language, caching, etc)

Can have data block for POST requests

▼ GET / HTTP/1.1

Host google.com

User-Agent HTTPie

▼ POST /api/recipe-steps HTTP/1.1

Content-Length 52

Content-Type application/json

Host localhost:8080

User-Agent HTTPie

```
▼ {  
  "step": "Another step 2",  
  "completed": true  
}
```

Anatomy of a Response



```
▼ HTTP/1.1 200 OK

Accept-Ranges      none
Cache-Control      private, max-age=0
Connection         close
Content-Security-Policy-Report-Only
  object-src 'none';base-uri 'self';script-src 'nonce-2jejjGsnQ79DqV
  g_1tEDsg' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-i
  nline' https: http::report-uri https://csp.withgoogle.com/csp/gws/
  other-hp
Content-Type       text/html; charset=ISO-8859-1
Date               Mon, 05 Feb 2024 04:39:01 GMT
Expires            -1
P3p                CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server             gws
Set-Cookie         1P_JAR=2024-02-05-04; expires=Wed, 06-Mar-2024 04:39:01 GMT; path
  =/; domain=.google.com; Secure, AEC=Ae3NU90wePlpJvyz38pnpz1Sqknrm
  Hf4He2-_o_p0MR1aBCIXXRKsT-eM0; expires=Sat, 03-Aug-2024 04:39:01 G
  MT; path=/; domain=.google.com; Secure; HttpOnly; SameSite=lax, NI
  D=511=kveNOT5fTmPF25XX3B3Y70t7hgHP0GiEVwkxgl09eiAa_0X4H9v9_oUgHGIp
  wcdJL5NuqKJmvoeP_7ngLm4iosTssjlXksdQMRjiFGmSc4AUHPdkNAIdw5-98RydmW
  4iVoSzFRAj7s5TcuBTUUhEbEUXo3BHBeB_p1hlT4ihv0s; expires=Tue, 06-Aug
  -2024 04:39:01 GMT; path=/; domain=.google.com; HttpOnly

Transfer-Encoding  chunked
Vary               Accept-Encoding
X-Frame-Options    SAMEORIGIN
X-Xss-Protection   0

▼ <!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="en"><head><meta
  content="Search the world's information, including webpages, images, videos and more. Google has
  many special features to help you find exactly what you're looking for." name="description"><meta
  content="noodp" name="robots"><meta content="text/html; charset=UTF-8" http-equiv="Content-Type">
  <meta content="/images/branding/googleg/1x/googleg_standard_color_128dp.png" itemprop="image">
```

```
▼ HTTP/1.1 201 Created

Connection         close
Content-Length      0
Date               Mon, 05 Feb 2024 04:45:08 GMT
```

Includes:

- Http version with status code
- Info about the data that's being sent in the response
- Data (ex: the html)

HTTP Status Codes



5 categories:

1xx -- informational

2xx -- successful

3xx -- redirection

4xx -- client error

5xx -- server error

Request Methods



Aka HTTP verbs

POST - to create/submit data entity

- Create

GET - requests a specific resource (only meant to retrieve data)

- Read

PUT - replace/update the existing entity

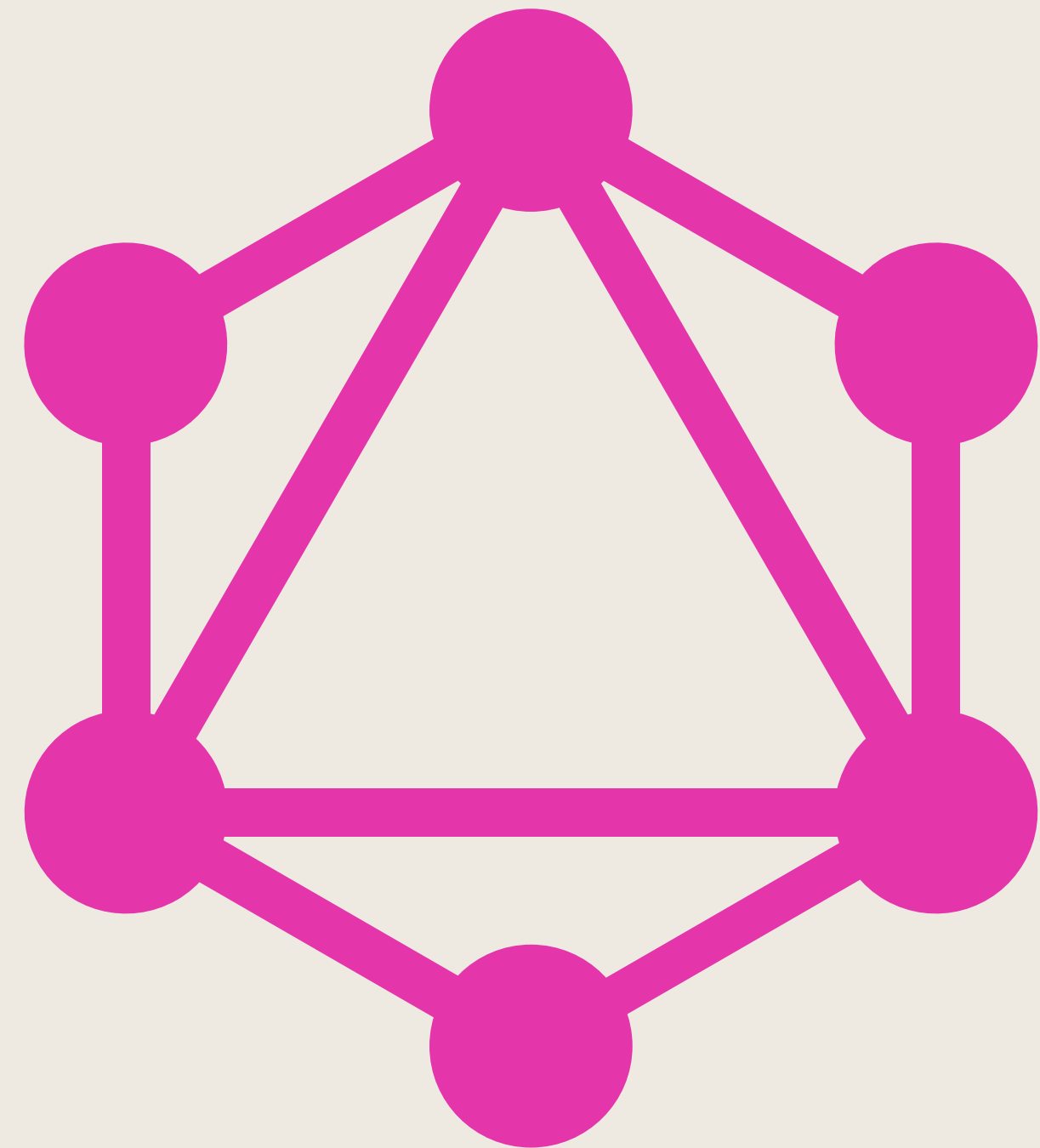
- Update

DELETE - deletes existing entity

- Delete

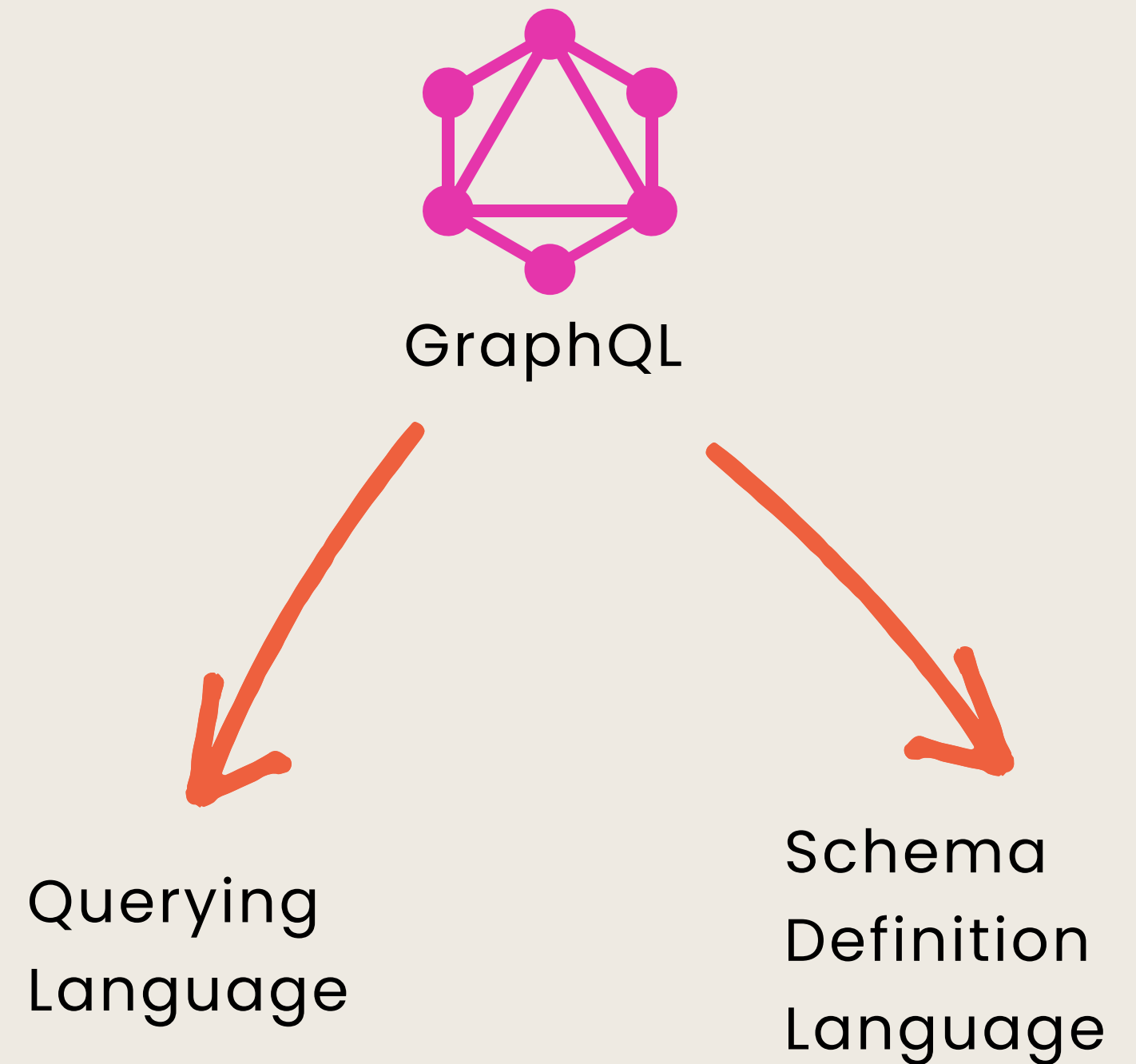
INTRO TO GRAPHQL

Week 6



WHAT IS GraphQL?

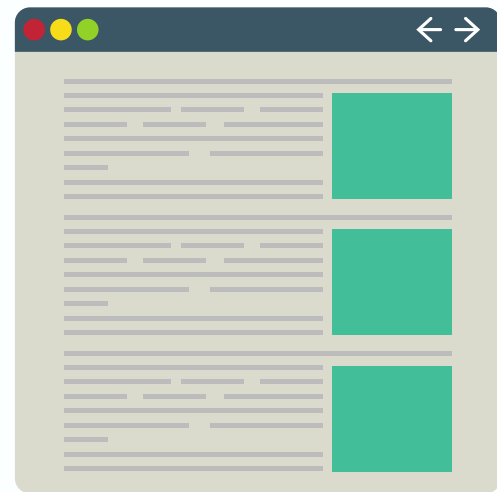
GraphQL is an API standard that is both a querying language and serverside runtime



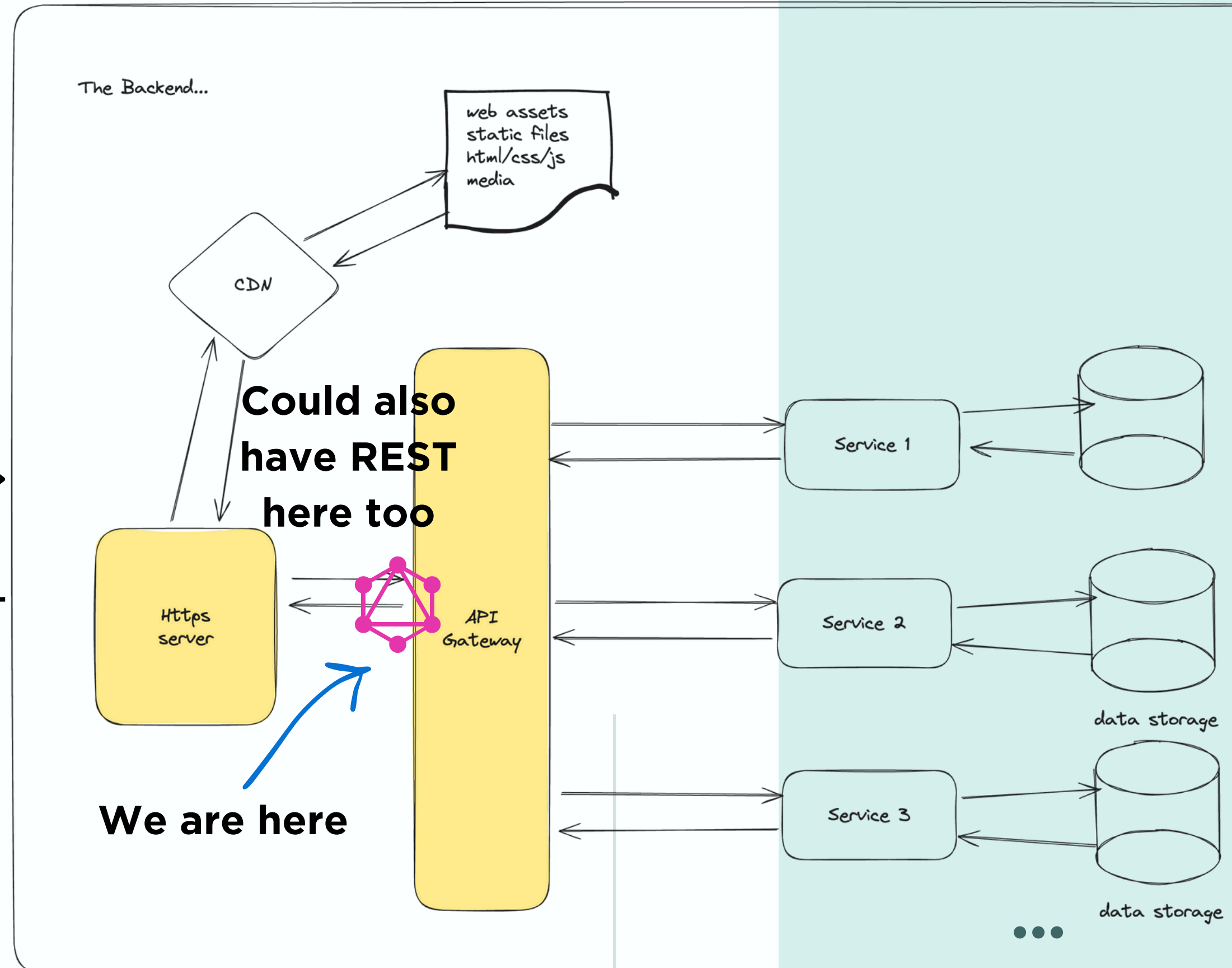
Patterns for the BE

Microservices

- When you break up a large app into smaller independent pieces
- Each service has one purpose



web browser
(client side)



WHY GRAPHQL?

Addresses the pitfalls of REST

Why NOT REST?

- Slow...
- Not great for aggregating multiple resources (data sources)
- Doesn't evolve well over time

WHY GRAPHQL?

Addresses the pitfalls of REST

Why NOT REST?

**No more over or
underfetching**



- Slow...
- Not great for aggregating multiple resources (data sources)
- Doesn't evolve well over time

WHY GRAPHQL?

Addresses the pitfalls of REST

Why NOT REST?

No more over or
underfetching

- Slow...
- Not great for aggregating multiple resources (data sources)
- Doesn't evolve well over time

**Uses one endpoint
to query/mutate
data**

WHY GRAPHQL?

Addresses the pitfalls of REST

Why NOT REST?

No more over or underfetching

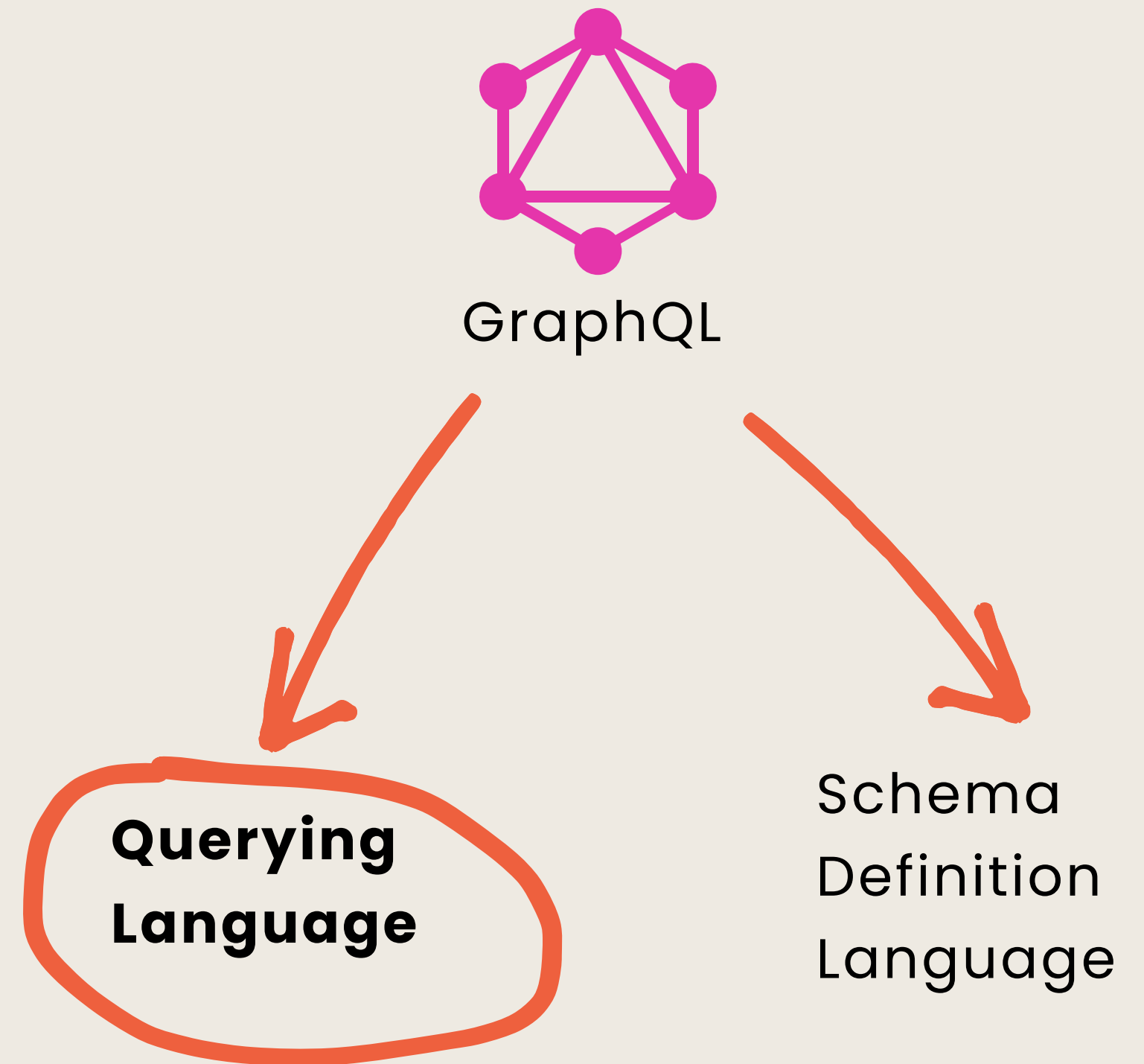
- Slow...
- Not great for aggregating multiple resources (data sources)
- Doesn't evolve well over time

Uses one endpoint to query/mutate data

Nothing changes in FE if we change the way queries are resolved in our BE services

WHAT IS GraphQL?

GraphQL is an API standard that is both a querying language and serverside runtime



QUERYING LANGUAGE

Queries allow us to **read** data (uses POST request)

```
query allFilms {  
  allFilms(first: 3) {  
    films {  
      title  
    }  
  }  
}
```

```
{  
  "data": {  
    "allFilms": {  
      "films": [  
        {  
          "title": "A New Hope"  
        },  
        {  
          "title": "The Empire Strikes Back"  
        },  
        {  
          "title": "Return of the Jedi"  
        }  
      ]  
    }  
  }  
}
```

Response is the same shape as the query

QUERYING LANGUAGE

Queries allow us to **read** data

operation
type



```
query allFilms {  
  allFilms(first: 3) {  
    films {  
      title  
    }  
  }  
}
```

QUERYING LANGUAGE

Queries allow us to **read** data

root object

operation
type



```
query allFilms {  
  allFilms(first: 3) {  
    films {  
      title  
    }  
  }  
}
```

The diagram shows a GraphQL query on a dark blue background. Two orange arrows point to specific parts of the query. One arrow points to the word 'query', which is highlighted in teal. The other arrow points to the opening curly brace of the 'allFilms' field, which is highlighted in orange. The query itself is written in a light blue monospace font. The structure is as follows: 'query allFilms {' followed by an indented 'allFilms(first: 3) {' followed by another indented 'films {' followed by an indented 'title' followed by closing braces. The 'allFilms' field's opening brace is highlighted in orange, and the 'query' keyword is highlighted in teal.

QUERYING LANGUAGE

Queries allow us to **read** data

root object

operation
type

```
query allFilms {  
  allFilms(first: 3) {  
    films {  
      title  
    }  
  }  
}
```

selected
field with
argument

QUERYING LANGUAGE

Queries allow us to **read** data

operation
type

root object

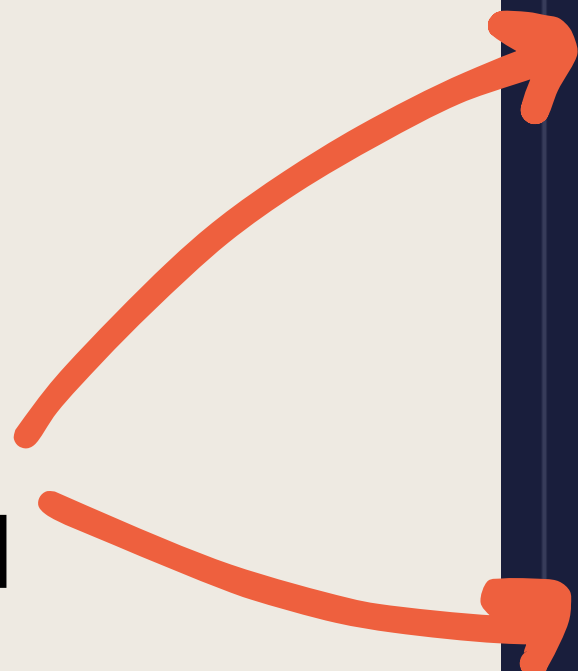
selected
field with
argument

sub-
selected
field

```
query allFilms {  
  allFilms(first: 3) {  
    films {  
      title  
    }  
  }  
}
```

QUERYING LANGUAGE

Aliases allow us to
query the same field
with different args



```
query allFilms {  
  firstThreeFilms: allFilms(first: 3) {  
    films {  
      title  
    }  
  }  
  
  lastThreeFilms: allFilms(last: 3) {  
    films {  
      title  
    }  
  }  
}
```

Star Wars API: **tinyurl.com/cs396starwars**

QUERYING LANGUAGE

Aliases allow us to query the same field with different args

```
{
  "data": {
    "firstThreeFilms": {
      "films": [
        {
          "title": "A New Hope"
        },
        {
          "title": "The Empire Strikes Back"
        },
        {
          "title": "Return of the Jedi"
        }
      ]
    },
    "lastThreeFilms": {
      "films": [
        {
          "title": "The Phantom Menace"
        },
        {
          "title": "Attack of the Clones"
        },
        {
          "title": "Revenge of the Sith"
        }
      ]
    }
  }
}
```

QUERYING LANGUAGE

Fragments are
reusable units to
construct queries

```
query allFilms {  
  firstThreeFilms: allFilms(first: 3) {  
    ...filmFields  
  }  
  
  lastThreeFilms: allFilms(last: 3) {  
    ...filmFields  
  }  
}  
  
fragment filmFields on FilmsConnection {  
  films {  
    title  
  }  
}
```

QUERYING LANGUAGE

Variables allow us to make query dynamic

```
{  
  "first": 2,  
  "last": 2  
}
```

```
query firstLastFilms($first: Int, $last: Int) {  
  firstThreeFilms: allFilms(first: $first) {  
    ...filmFields  
  }  
  
  lastThreeFilms: allFilms(last: $last) {  
    ...filmFields  
  }  
}  
  
fragment filmFields on FilmsConnection {  
  films {  
    title  
  }  
}
```

QUERYING LANGUAGE

Mutations allow us to
modify data

```
mutation AddRecipeStep {  
  addRecipeStep(step: "New Step", completed:false) {  
    step,  
    completed  
  }  
}
```

QUERIES VS MUTATIONS

Mutations run
sequentially (no race
conditions)

Queries run in **parallel**

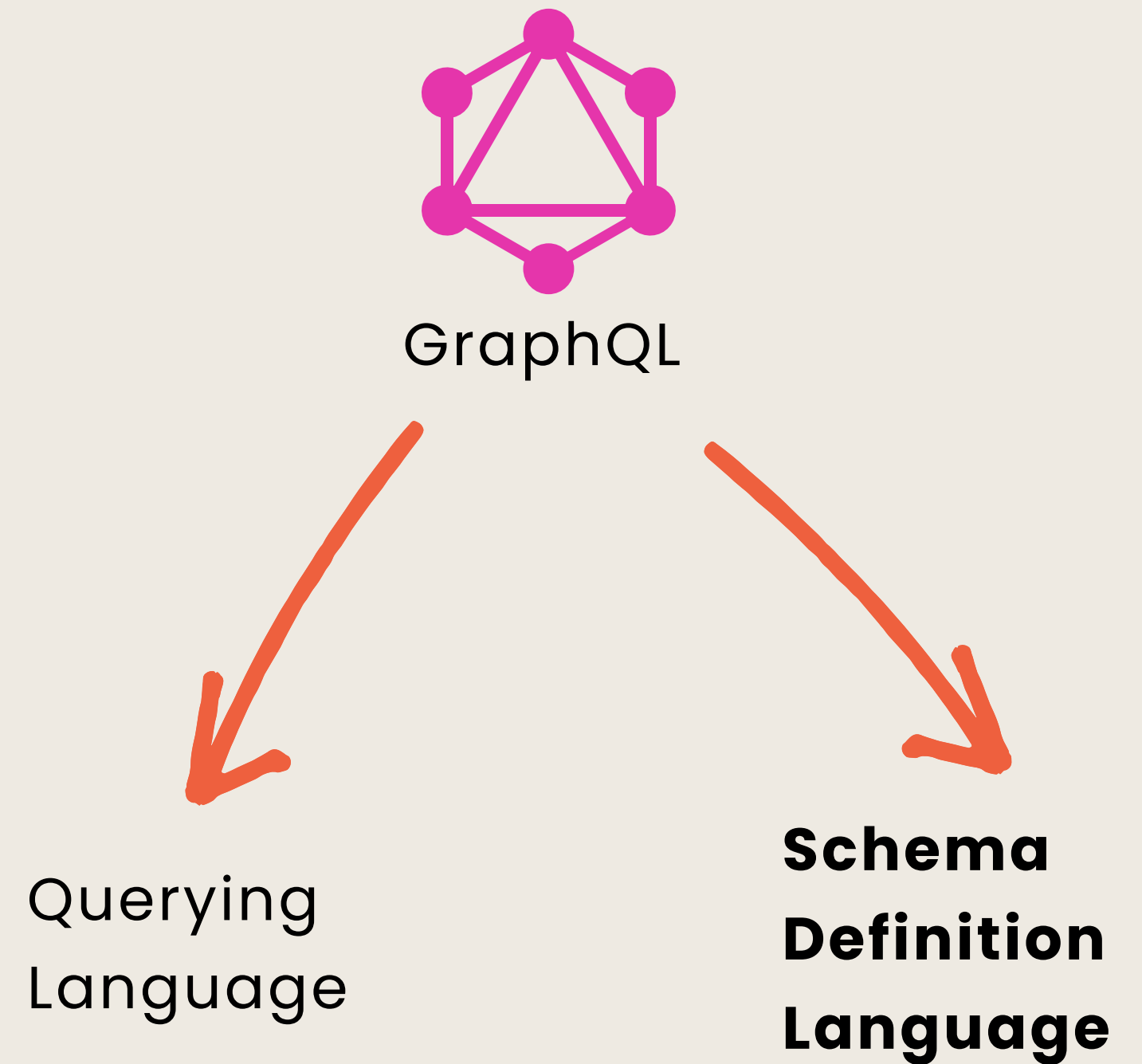
ExecuteRequest(schema, document, operationName, variableValues, initialValue) :

1. Let *operation* be the result of *GetOperation(document, operationName)*.
2. Let *coercedVariableValues* be the result of *CoerceVariableValues(schema, operation, variableValues)*.
3. If *operation* is a query operation:
 - a. Return *ExecuteQuery(operation, schema, coercedVariableValues, initialValue)*.
4. Otherwise if *operation* is a mutation operation:
 - a. Return *ExecuteMutation(operation, schema, coercedVariableValues, initialValue)*.
5. Otherwise if *operation* is a subscription operation:
 - a. Return *Subscribe(operation, schema, coercedVariableValues, initialValue)*.

From: <https://spec.graphql.org/June2018/#sec-Executing-Requests>

WHAT IS GraphQL?

GraphQL is an API standard that is both a querying language and serverside runtime



SCHEMA

Schemas are a contract between the client and the server on how the client can access data

```
schema {  
  query: Query  
  mutation: Mutation  
}  
  
type Query {  
  recipeSteps: [RecipeStep]  
  recipeStep(id: ID!): RecipeStep  
}  
  
type Mutation {  
  addRecipeStep(step: String, completed: Boolean): RecipeStep  
}  
  
type RecipeStep {  
  id: ID  
  step: String  
  completed: Boolean  
}
```

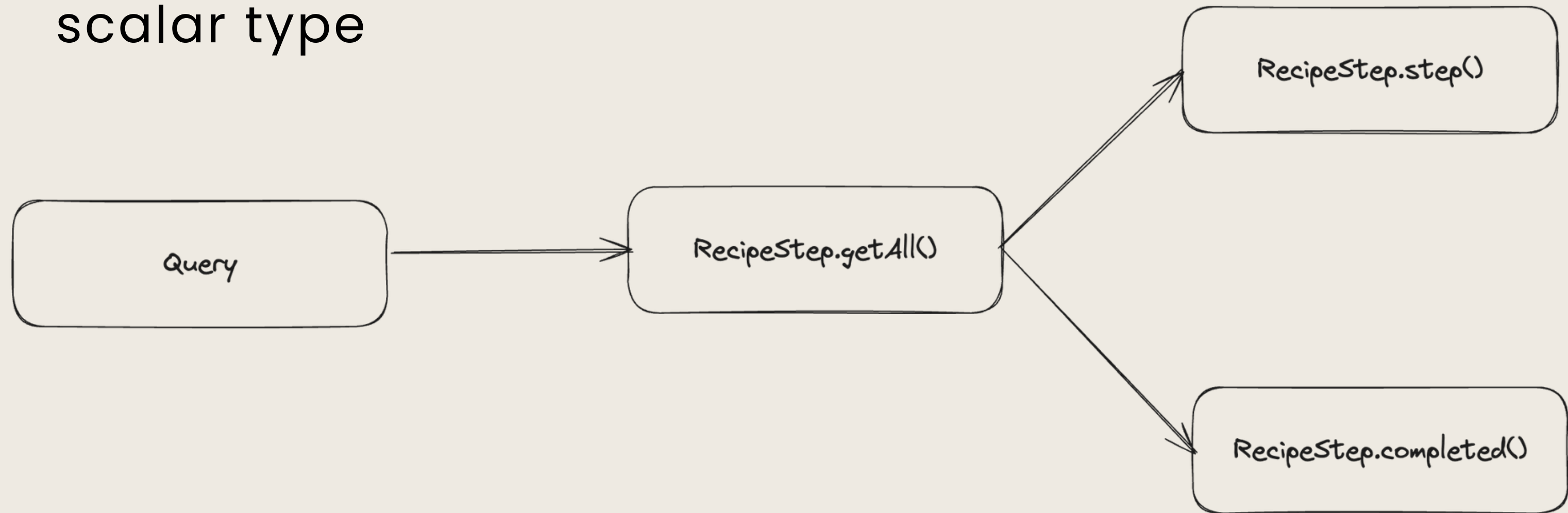
TYPES

- Query, Mutation, and Subscription are entry point types
- Most are object types
- Scalar types are your classic primitives

```
schema {  
  query: Query  
  mutation: Mutation  
}  
  
type Query {  
  recipeSteps: [RecipeStep]  
  recipeStep(id: ID!): RecipeStep  
}  
  
type Mutation {  
  addRecipeStep(step: String, completed: Boolean): RecipeStep  
}  
  
type RecipeStep {  
  id: ID  
  step: String  
  completed: Boolean  
}
```

RESOLVERS

- The functions that will return a value for the corresponding field
- All object types must resolve to a scalar type



PROJECT INTROS